5.4 扩展：余弦相似度与大模型的底层逻辑

在当今最火热的人工智能领域，无论是处理文本的大语言模型，还是那些能看图说话的视觉语言模型（VLM），它们每天都在处理海量的文本和图像。机器不懂人类的语言，更不懂风花雪月，它们只能把万事万物都转换为高维空间里的一串数字。我们称之为“向量”。

这时候，怎么衡量两段文本的语义是否相近？怎么判断一张图片和一句话是否匹配？答案是大模型时代最基本的数学工具——余弦相似度。

5.4.1 从相关系数到“余弦相似度”

我们在上文已经对相关系数和余弦相似度的关系做了简单的介绍。我们用具体的例子来看看它们到底是怎么分道扬镳的。

首先回顾一个核心结论：我们说相关系数是两个“中心化”向量夹角的余弦。如果我们不做中心化，直接看原始数据拼成的向量呢？得到的就是“**余弦相似度**”。

为什么要抛弃“中心化”？在很多真实的业务场景里，数据的绝对数值本身就带有强烈的物理意义或者语义特征。例如，在一张黑白照片里，像素值为 0 代表纯黑，255 代表纯白；在文本分析里，某个词出现的次数是 0 次还是 5 次，这是非常关键的频次信息。如果我们像计算相关系数那样，强行把所有像素或者词频都减去一个平均值，这就好比把所有大楼都连根拔起、统一挪到同一条起跑线上——相对高矮没变，但每栋楼原来站在哪里的信息就丢了。

这时候，我们不再关心它们各自围绕平均值是怎么波动的，我们只关心：这两个原始的箭头，在空间中是不是指向同一个大方向。

定义：对于两个非零向量 $\boldsymbol{u}$ 和 $\boldsymbol{v}$，它们的余弦相似度定义为它们夹角的余弦值。

$$\text{余弦相似度}(\boldsymbol{u},\boldsymbol{v}) = \frac{\boldsymbol{u}\cdot\boldsymbol{v}}{\|\boldsymbol{u}\|\|\boldsymbol{v}\|} = \cos\phi$$

其中 $\boldsymbol{u}\cdot\boldsymbol{v}$ 是向量的内积，$\|\boldsymbol{u}\|$ 和 $\|\boldsymbol{v}\|$ 分别是向量的模长，ϕ 是两个向量的夹角。

作为对比，我们再把相关系数的公式摆在旁边。

$$\rho_{XY} = \frac{\mathrm{Cov}(X,Y)}{\sigma_X\sigma_Y} = \frac{E[(X-\mu_X)(Y-\mu_Y)]}{\sqrt{E[(X-\mu_X)^2]}\sqrt{E[(Y-\mu_Y)^2]}}$$

如果将 X 和 Y 的观测值看作向量，那么相关系数就是先将每个向量减去其均值（中心化），再计算这两个新向量的余弦相似度。所以，**相关系数可以看作一种“先中心化，再计算余弦相似度”的特殊操作**。

举个具体的数值例子来展示这两个概念的细微差别。假设我们有两个变量 X 和 Y，它们的三次观测值为

$$X:1,2,3。 \quad Y:2,4,6。$$

我们将这些观测值视为空间里的两个箭头（向量）：$\boldsymbol{x}=(1,2,3)$，$\boldsymbol{y}=(2,4,6)$。

先来计算协方差和相关系数：假设每个观测等概率（权重均为 $1/3$）。

$$
\begin{aligned}
E[X] &= \frac{1+2+3}{3} = 2, \quad E[Y] = \frac{2+4+6}{3} = 4 \\
E[XY] &= \frac{1\times 2 + 2\times 4 + 3\times 6}{3} = \frac{2+8+18}{3} = \frac{28}{3} \\
\mathrm{Cov}(X,Y) &= E[XY] - E[X]E[Y] = \frac{28}{3} - 2\times 4 = \frac{28}{3} - \frac{24}{3} = \frac{4}{3} \\
\mathrm{Var}(X) &= E[(X-2)^2] = \frac{(1-2)^2 + (2-2)^2 + (3-2)^2}{3} = \frac{1+0+1}{3} = \frac{2}{3} \\
\mathrm{Var}(Y) &= E[(Y-4)^2] = \frac{(2-4)^2 + (4-4)^2 + (6-4)^2}{3} = \frac{4+0+4}{3} = \frac{8}{3} \\
\rho_{XY} &= \frac{\mathrm{Cov}(X,Y)}{\sqrt{\mathrm{Var}(X)\mathrm{Var}(Y)}} = \frac{4/3}{\sqrt{(2/3)\times(8/3)}} = \frac{4/3}{\sqrt{16/9}} = \frac{4/3}{4/3} = 1
\end{aligned}
$$

再来计算余弦相似度（使用原始向量，不中心化）。

$$
\begin{aligned}
\boldsymbol{x}\cdot\boldsymbol{y} &= 1\times 2 + 2\times 4 + 3\times 6 = 2+8+18 = 28 \\
\|\boldsymbol{x}\| &= \sqrt{1^2+2^2+3^2} = \sqrt{14} \\
\|\boldsymbol{y}\| &= \sqrt{2^2+4^2+6^2} = \sqrt{56} = 2\sqrt{14} \\
\text{余弦相似度} &= \frac{28}{\sqrt{14}\times 2\sqrt{14}} = \frac{28}{2\times 14} = 1
\end{aligned}
$$

在这个例子中，闭上眼睛想象一下，在三维空间里，向量 $\boldsymbol{y}$ 其实就是向量 $\boldsymbol{x}$ 沿着原方向拉长了一倍。它们指向绝对一致的方向，夹角为 0，所以余弦相似度为 1。同时因为是完美的正比例关系，相关系数也是 1。

但是，现实哪有这么完美？如果我们稍微给数据掺点沙子，换一组数据，例如 $X=[1,2,3]$，$Y=[2,4,5]$，我们完整手算一遍协方差、相关系数和余弦相似度，看看会有什么不同。

（1）**计算协方差和相关系数：**假设每个观测等概率（权重均为 $1/3$）。

$$
\begin{aligned}
E[X] &= \frac{1+2+3}{3} = 2, \quad E[Y] = \frac{2+4+5}{3} = \frac{11}{3} \approx 3.6667 \\
E[XY] &= \frac{1\times 2 + 2\times 4 + 3\times 5}{3} = \frac{2+8+15}{3} = \frac{25}{3} \approx 8.3333 \\
\mathrm{Cov}(X,Y) &= E[XY] - E[X]E[Y] = \frac{25}{3} - 2\times\frac{11}{3} = \frac{25}{3} - \frac{22}{3} = \frac{3}{3} = 1 \\
\mathrm{Var}(X) &= E[(X-2)^2] = \frac{(1-2)^2 + (2-2)^2 + (3-2)^2}{3} = \frac{1+0+1}{3} = \frac{2}{3} \approx 0.6667 \\
\mathrm{Var}(Y) &= E[(Y-\tfrac{11}{3})^2] = \frac{(2-\frac{11}{3})^2 + (4-\frac{11}{3})^2 + (5-\frac{11}{3})^2}{3} \\
&= \frac{(-\frac{5}{3})^2 + (\frac{1}{3})^2 + (\frac{4}{3})^2}{3} = \frac{\frac{25}{9}+\frac{1}{9}+\frac{16}{9}}{3} = \frac{\frac{42}{9}}{3} = \frac{14}{9} \approx 1.5556 \\
\rho_{XY} &= \frac{\mathrm{Cov}(X,Y)}{\sqrt{\mathrm{Var}(X)\mathrm{Var}(Y)}} = \frac{1}{\sqrt{\frac{2}{3}\times\frac{14}{9}}} = \frac{1}{\sqrt{\frac{28}{27}}} = \sqrt{\frac{27}{28}} \approx 0.982
\end{aligned}
$$

（2）**计算余弦相似度（使用原始向量，不中心化）。**将观测值视为向量：$\boldsymbol{x}=(1,2,3)$,

$\boldsymbol{y} = (2, 4, 5)$。

$$\begin{aligned}
\boldsymbol{x} \cdot \boldsymbol{y} &= 1 \times 2 + 2 \times 4 + 3 \times 5 = 2 + 8 + 15 = 25 \\
\|\boldsymbol{x}\| &= \sqrt{1^2 + 2^2 + 3^2} = \sqrt{14} \approx 3.7417 \\
\|\boldsymbol{y}\| &= \sqrt{2^2 + 4^2 + 5^2} = \sqrt{4 + 16 + 25} = \sqrt{45} \approx 6.7082 \\
\text{余弦相似度} &= \frac{\boldsymbol{x} \cdot \boldsymbol{y}}{\|\boldsymbol{x}\|\|\boldsymbol{y}\|} = \frac{25}{\sqrt{14} \times \sqrt{45}} = \frac{25}{\sqrt{630}} \approx \frac{25}{25.0998} \approx 0.996
\end{aligned}$$

看看这个结果：**相关系数约为 0.982，而余弦相似度约为 0.996**。

为什么会有这种细微的不同？因为在这个例子里，数据没有经过中心化处理，且 Y 也并不完全是 X 的等比例放大。这就好比考察两个人在山野里徒步：相关系数看的是他们“走路步伐的起伏节奏是不是一致”，而余弦相似度看的是他们“最终前进的绝对经纬度方向是不是重合”。

当你把数据的均值归零（也就是把他们拉到同一条起跑线上做中心化操作）之后，步伐的节奏和前进的方向就变成同一码事了。这也就是我们前面强调的：当数据已经中心化时，相关系数就严格等于余弦相似度。

5.4.2 余弦相似度的具体应用：搜索与推荐系统

掌握了上面的公式，你可能会觉得：这不就是把几个数字乘一乘、加一加、开个根号吗？这么枯燥的计算，到底有多大用处？

只要把这些数字赋予实际的业务意义，事情就有意思了。在许多领域，我们喜欢把具象的事物变成抽象的向量，一旦变成了向量，万事万物就可以用数学工具来算它们的关系了。

例如，我们试着用最简单的两个维度来刻画一本书。

- **维度 1**：情节戏剧性（$0 \sim 10$ 分，分数越高越狗血刺激）。
- **维度 2**：知识信息量（$0 \sim 10$ 分，分数越高越硬核烧脑）。

在这样一个二维坐标系里，假设一本武侠小说《刀光剑影》是 $(9, 1)$，而一本历史演义《王朝风云》是 $(8, 2)$，一本学术专著《深度思考》则是 $(1, 9)$。

假设你周末刚熬夜读完了《刀光剑影》（向量 A），系统想要趁热打铁，给你推荐一本最相似的书。作为推荐算法背后的“上帝”，你该推荐《王朝风云》（向量 B）还是《深度思考》（向量 C）？

这个时候就需要掏出**余弦相似度**这把尺子——也就是直接计算两个原始向量夹角的余弦——来看看它们的匹配度。

- **相似度** $(A, B) = (9 \times 8 + 1 \times 2)/(\sqrt{9^2 + 1^2} \times \sqrt{8^2 + 2^2}) \approx 0.99$。
- **相似度** $(A, C) = (9 \times 1 + 1 \times 9)/(\sqrt{82} \times \sqrt{82}) = 18/82 \approx 0.22$。

书籍在二维语义空间中的向量表示如图5-9所示。结果一目了然，A 和 B 极其相似。为什么？因为余弦相似度是一个只看“方向”不看“长短”的度量工具。即使《王朝风云》的戏剧性绝对评分比《刀光剑影》少了一分，但它们在图纸上都坚定地指向了“高情节、低知识”的那个偏激角落。系统判定你们志趣相投，马上就会把《王朝风云》推到你的首页。余弦相似度只关注向量的夹角（方向），A 与 B 夹角极小，因此被判定为极度相似，而忽略了它们在绝对长度上的细微差异。

这种把物品变成向量、再算夹角余弦的操作，在很多互联网产品中都有应用。

- **推荐系统**：不仅仅是书，你这个人，以及你看过的电影、买过的鞋，都被映射为高维向量。淘宝或抖音的系统每秒钟都在寻找与你喜好向量方向最一致（余弦相似度最高）的商品或短视频推荐给你。
- **人脸识别**：你手机里的人脸解锁是怎么工作的？摄像头扫过你的脸，底层的神经网络瞬间提取出五官比例、骨骼走向等几百个特征，生成一个属于你的专属向量。当你每次解锁时，系统并不要求这次的几百个数字和最初录入时一模一样（毕竟光线变了，你还可能戴了眼镜），它只要计算当前脸部向量和安全芯片里主人向量之间的余弦相似度。只要这个值超过设定的门槛，锁就开了。
- **搜索引擎**：你的查询词汇和互联网上的每一个网页，都被变成了一个巨大的词频向量。系统计算它们之间的余弦相似度，把方向最匹配（即最相关）的网页排在最前面。

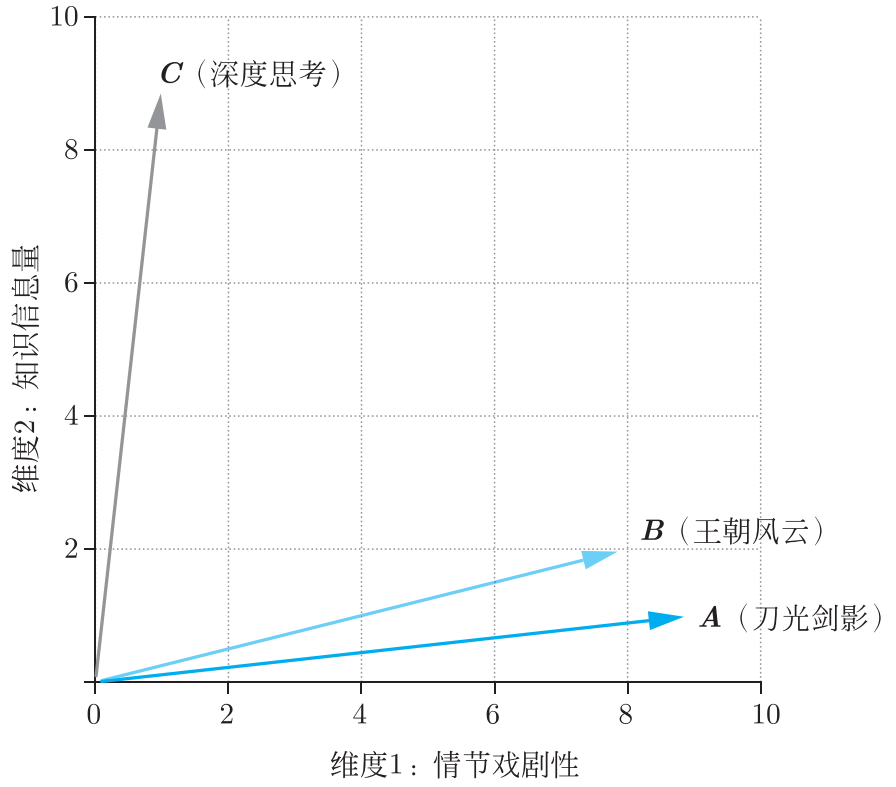


图 5-9: 书籍在二维语义空间中的向量表示

来看一个每天都在发生的真实场景。

我们假设大模型提取了以下三个极其抽象的隐式维度。

- **维度 1：IT 与算法属性**（$0 \sim 1$，越高代表越偏向计算机、代码和人工智能）。
- **维度 2：农业与自然属性**（$0 \sim 1$，越高代表越偏向植物种植、大自然生态）。
- **维度 3：商业与变现实操**（$0 \sim 1$，越高代表越偏向如何赚钱、如何落地实操）。

现在，你在搜索框输入了极其精练的几个字：“**苹果机器学习实战**”。系统底层的神经网络瞬间吃透了你的潜台词：你是想找科技巨头苹果公司（Apple）相关的 AI 算法写代码教程。它将这句查询转换为了一个向量：$\boldsymbol{q} = (0.9, 0.0, 0.4)$。意思是：IT 浓度极高，跟种地毫无关系，带有一些实操落地属性。

此时，互联网上有两篇候选文章。

- **文档 A** 为《iOS 平台上的神经网络与 CoreML 开发指南》。这篇文章的向量化特

征是 $\boldsymbol{a} = (1.0, 0.0, 0.3)$。

- **文档 B** 为《果园实战：苹果树的机器化种植与学习指南》。这篇文章的向量化特征是 $\boldsymbol{b} = (0.1, 0.9, 0.2)$。

如果你用十年前的老古董搜索引擎，它会惊喜地发现文档 B 的标题里完美包含了“苹果”“实战”“机器”“学习”这几个字，字面命中率高达 100%，绝对会把它推到第一位。而文档 A 连这些字的影子都没有。

但是，现代 AI 是怎么做的？它用的是**余弦相似度**（即计算 $\cos\phi = \frac{\boldsymbol{q}\cdot\boldsymbol{x}}{\|\boldsymbol{q}\|\|\boldsymbol{x}\|}$）。

算出三个向量的模长（空间长度）。

$$\|\boldsymbol{q}\| = \sqrt{0.9^2 + 0.0^2 + 0.4^2} = \sqrt{0.81 + 0 + 0.16} = \sqrt{0.97} \approx 0.985$$
$$\|\boldsymbol{a}\| = \sqrt{1.0^2 + 0.0^2 + 0.3^2} = \sqrt{1.0 + 0 + 0.09} = \sqrt{1.09} \approx 1.044$$
$$\|\boldsymbol{b}\| = \sqrt{0.1^2 + 0.9^2 + 0.2^2} = \sqrt{0.01 + 0.81 + 0.04} = \sqrt{0.86} \approx 0.927$$

接着，计算查询词与两篇文章的余弦相似度。

- **查询 $\boldsymbol{q}$ 与文档 A（字面完全不匹配）的相似度**。

$$\frac{0.9 \times 1.0 + 0.0 \times 0.0 + 0.4 \times 0.3}{0.985 \times 1.044} = \frac{0.9 + 0 + 0.12}{1.028} = \frac{1.02}{1.028} \approx 0.992$$

- **查询 $\boldsymbol{q}$ 与文档 B（字面完美命中）的相似度**。

$$\frac{0.9 \times 0.1 + 0.0 \times 0.9 + 0.4 \times 0.2}{0.985 \times 0.927} = \frac{0.09 + 0 + 0.08}{0.913} = \frac{0.17}{0.913} \approx 0.186$$

那篇包含了“苹果”“机器”“学习”的果园种树指南（文档 B），因为在多维空间里的“大方向”指向了农业，和你的查询意图南辕北辙，余弦相似度只有可怜的 0.186，直接被丢进垃圾桶。

而那篇压根没提搜索词，但讲的全是 iOS 算法代码的文档 A，在这个抽象空间里与你的意图几乎完全重合（相似度高达 0.992），被稳稳地推到了你面前。

底层的神经网络负责把复杂的概念剥离字面，变成方向明确的向量；而余弦相似度负责判断这些向量的方向是否一致。人工智能能够“懂你”，靠的就是这套机制。

5.4.3 余弦相似度在大模型中的应用

在我们刚才的例子里，维度只有区区两三个。而在真实的大语言模型（LLM）背后，这种计算被推向了远超直觉的规模。

如果你在工作中经常使用大语言模型写代码或做翻译，你可能会好奇，它们一堆硅芯片，怎么能在毫秒级的时间内读懂你的意思？其实，它们在拿到你输入的句子时，做的第一件事叫作“词嵌入”（Embedding）。

无论是英文单词还是中文汉字，模型都会把词典里的每一个词，投射到一个高达数百甚至数千维的超高维空间里。在这个空间里，词与词之间有了具体的“距离”和“方向”。

这里有一个自然语言处理领域经典的例子（见图 5-10）：如果你在那个高维空间里，把代表“国王”的向量减去“男人”的向量，再加上“女人”的向量，最后得到的那个坐标位置，离它最近的词汇向量竟然是“皇后”。这说明什么？这说明在高维空间里，模型不仅记住了字怎么写，还学会了诸如“性别偏移”、“皇权属性”这些极其抽象的语义概念。相似的语义会自动聚拢，甚至还能像物理学中的力一样进行加减运算。

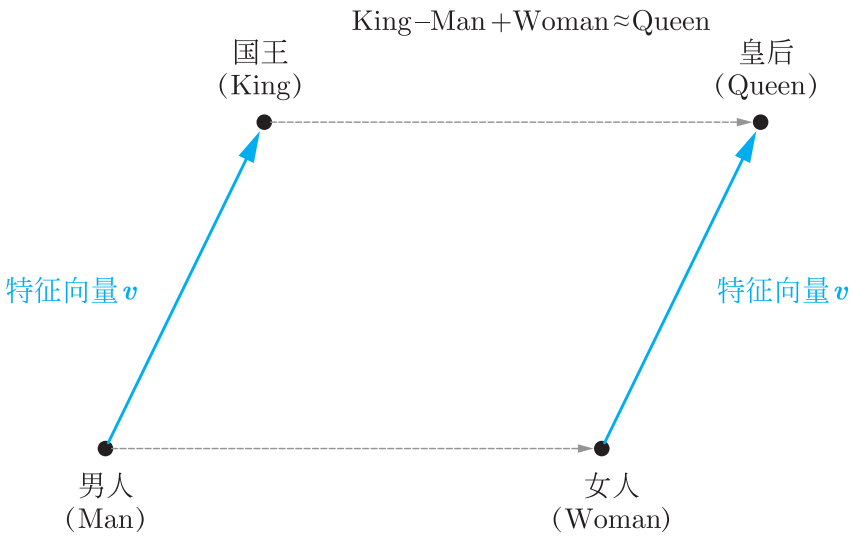


图 5-10: 大模型潜空间中的语义算术

到了多模态视觉语言模型（VLM）这边，事情又不一样了。

当你发给 AI 一张“正在奔跑的橘猫”的照片，并问它图里有什么时，模型是怎么将毫不相干的像素矩阵和人类文字挂钩的?

这种做法非常暴力。它内部有两个编码器，一个负责把图片压扁成一个例如 768 维的向量，另一个负责把文字也转换成同等维度的 768 维向量。在日复一日的残酷训练中，工程师们用海量的图文数据强迫它：只要图片和文字是匹配的，就去调整内部参数，让这两个向量的夹角尽可能小；如果不匹配，就让夹角尽可能大。

于是，大功告成。当你在上千万张海量图库里搜索“橘猫”时，AI 根本不需要去每一张图里做肉眼般的像素比对。它只需要把你输入的文字“橘猫”瞬间变成一个 768 维的向量，然后调用 GPU 飞速地计算这个文本向量与图库里所有图片向量的**余弦相似度**。谁的相似度最接近 1，谁就是你要找的那张照片。

在成千上万个维度的空间里，余弦相似度根本不是什么高深莫测的魔法。它帮我们自动过滤掉了句子长短、图片亮暗、词频大小带来的数值干扰，专门用来测量两个事物在语义方向上的一致程度。

5.4.4 Python 实验室



Python 实验室

建议你打开配套资源包中的 `cosine_similarity_search_demo.py`。这段脚本分两步：先用同一组向量分别计算原始余弦相似度和中心化后的余弦相似度，验证相关系数可以看作“先中心化、再计算余弦相似度”；再用三个隐式语义维度搭一个极简搜索引擎，比较“iOS 代码教程”“果园机械化种植”“AI 大模型变现实战”与查询向量的匹配度。

你可以先把 `QUERY_VECTOR` 从 `[0.9, 0.0, 0.4]` 改成 `[0.1, 0.9, 0.1]`，观察搜索结果如何从科技文档转向农业文档；也可以修改 `VECTOR_X`、`VECTOR_Y`，感受原始方向

相似和中心化后线性相关之间的差别。这个实验要看的不是公式本身有多难，而是“把对象变成向量，再比较方向”这件事如何支撑搜索、推荐和大模型语义匹配。

5.4.5 总结

这一节一直围绕一个问题：两个东西到底像不像?

在二维平面里，我们可以看两个箭头的夹角；到了推荐系统、搜索引擎和大模型里，书、网页、句子、图片也可以被变成一串数字，放进更高维的空间。方向越接近，系统越倾向于认为它们相似。余弦相似度做的就是这件事。

相关系数也可以放到这个视角下理解：先把数据拉回各自的中心，再比较它们一起变化的方向。当一个对象能被表示成向量时，我们就可以用方向来衡量它们的相似性。这就是余弦相似度在搜索、推荐和大模型里的基本作用。